STP-Based Layer 2 Attacks

IE3122: Network Security | Week 4

Spanning Tree Protocol: function, attacks, and hardening

Today's Agenda

- Why STP matters, and what it does
- STP functionality and the algorithm, including how path cost decides port roles
- Different versions of STP
- Attacks possible based on STP
- Mitigations: Root Guard, BPDU Guard, BPDU Filter, Loop Guard, BackboneFast, UplinkFast
- Where each mitigation is actually deployed

Learning Outcomes for Today

LO1

- Describe common network vulnerabilities and attacks, today specifically STP manipulation.

LO2

- Evaluate different mitigation techniques for vulnerabilities and make recommendations.

- *By the end of this session you should be able to explain how STP actually decides its topology, how that process can be abused, and which hardening feature belongs on which port.*

STP Fundamentals

Why redundant links need a traffic cop

Why STP Matters

- Real networks build in redundant physical links between switches on purpose. If one link fails, traffic should still have a path.
- But Ethernet frames have no built-in ‘hop limit’ the way IP packets do (recall Week 1's TTL field). A frame on a looped Layer 2 topology can circulate forever.
- Without something managing this, redundant links cause broadcast storms, duplicate frame delivery, and MAC address table instability. A loop can take down an entire switched network within seconds.
- Spanning Tree Protocol (STP, IEEE 802.1D) exists to solve exactly this: keep the redundancy, but mathematically guarantee there's no active loop.

What STP Actually Does

- STP has every participating switch exchange special frames called BPDUs (Bridge Protocol Data Units) to agree on a loop-free logical topology.
- It doesn't remove redundant links physically. It logically blocks just enough of them so that exactly one active path exists between any two switches.
- If an active link fails, STP recalculates and can bring a previously blocked link into use. Redundancy is preserved, just not simultaneously active.
- This blocking decision is entirely trust-based: any switch that speaks BPDUs convincingly gets treated as legitimate, the root of today's attacks.

STP Functionality & the Algorithm

How switches agree on a loop-free topology

The STP Algorithm, in Three Steps

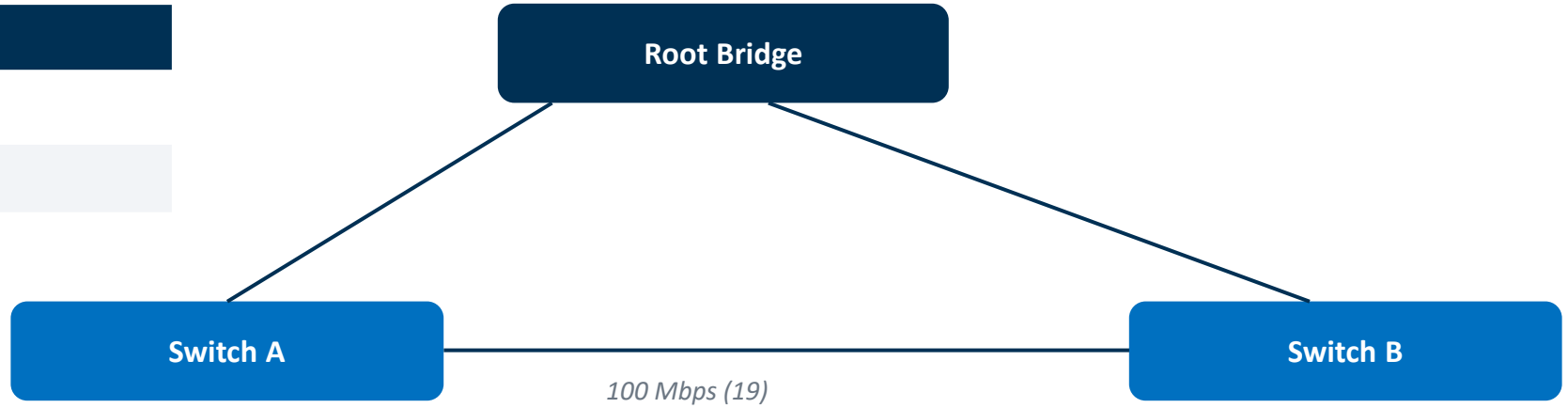
- 1. Elect a Root Bridge: every switch compares Bridge IDs. The switch with the lowest ID becomes the single reference point the whole topology is built around.
- 2. Elect a Root Port on every other switch: the port with the lowest-cost path back to the root bridge.
- 3. Elect a Designated Port on every network segment: the port that will forward traffic onto that segment. Every other port on that segment blocks.
- The result: exactly one loop-free path from every switch to the root, with all other links present but not actively forwarding.

Root Bridge Election

- Every switch has a Bridge ID: a 2-byte priority (default 32768, configurable in increments of 4096) followed by the switch's MAC address.
- Switches flood BPDUs advertising their own Bridge ID; the switch advertising the numerically lowest Bridge ID wins and becomes the root bridge.
- Priority is compared first; MAC address only breaks a tie. This matters a great deal for today's attack: whoever advertises the lowest priority wins, regardless of whether they should.
- In a well-designed network, the administrator deliberately lowers priority on the intended root (usually a core/distribution switch) so the election isn't left to chance.

Calculating Path Cost and Choosing Ports

Link Speed	Cost
10 Mbps	100
100 Mbps	19
1 Gbps	4



- Root Path Cost is the sum of every link's cost along a path back to the root. Switch A's options: direct link, cost 19; via B, 4 + 19 = 23. Its Root Port is the direct link, since 19 is lower.
- Switch B's options: direct link, cost 4; via A, 19 + 19 = 38. Its Root Port is the direct link, cost 4.
- On the A-to-B segment: A advertises its own root path cost of 19; B advertises 4. B's cost is lower, so B's port becomes Designated on that segment, and A's port becomes Non-Designated (blocking), even though A already has a working Root Port elsewhere.

Port Roles

Role	What it does
Root Port	The one port on a non-root switch with the best path back to the root bridge; forwards traffic
Designated Port	The forwarding port on each network segment, one per segment
Non-Designated (Blocking) Port	Any other port on a segment; receives BPDUs but does not forward data traffic

Port States

State	What's happening
Blocking	Receives BPDUs only; does not forward frames or learn MAC addresses
Listening	Preparing to forward; processing BPDUs, still not learning MACs or forwarding
Learning	Building the MAC address table; still not forwarding data frames
Forwarding	Fully operational; forwarding frames and learning MAC addresses

Classic 802.1D STP takes up to 50 seconds to converge through these states. Slow enough that it became the main motivation for the newer versions we cover next.

Versions of STP

The same core algorithm, made faster and more flexible

STP Versions at a Glance

Version	Standard	Key characteristic
STP	IEEE 802.1D	Original version; one spanning tree for the entire network; slow convergence (~50s)
RSTP (Rapid STP)	IEEE 802.1w	Adds new port roles (Alternate, Backup) and a handshake mechanism for fast convergence (seconds, not tens of seconds)
PVST+ / Rapid PVST+	Cisco proprietary	Runs one independent spanning tree instance per VLAN, allowing per-VLAN load balancing across redundant links
MSTP (Multiple STP)	IEEE 802.1s	Maps multiple VLANs to a smaller number of spanning tree instances, more scalable than one instance per VLAN

What RSTP Actually Improves

- Adds two new port roles beyond STP's root/designated/blocking: Alternate Port (a backup path to the root, ready to take over) and Backup Port (a backup for a designated port on the same segment).
- Because these backup roles already know their standby role in advance, RSTP can transition to forwarding almost immediately after a failure. No need to wait through Listening and Learning.
- This is why the mitigations we cover shortly (BackboneFast, UplinkFast) were originally designed as separate Cisco features to patch STP's slow convergence. RSTP later absorbed much of what they did into the core protocol.

STP-Based Attacks

Exploiting a protocol that trusts every BPDU it receives

Why STP Can Be Attacked

- BPDUs carry no authentication. Any device connected to a switch port can send them, and the switch has no way to verify the sender is a legitimate switch at all.
- The entire root bridge election is just ‘whoever claims the lowest priority wins’. There's no check on whether that claim is trustworthy.
- This is the same theme we've seen all module: a protocol designed for a trusted, physically-controlled network breaks down once an attacker can plug into an access port.

Three STP Attack Patterns

- Root Bridge Manipulation (Root Bridge Spoofing): an attacker convinces the network to accept their device as the root bridge, redirecting the topology through them.
- Topology Change (TC) Flooding / Spoofing: an attacker abuses STP's topology-change notification mechanism to repeatedly force network-wide MAC address table flushes.
- STP Denial of Service (DoS): an attacker floods the network with BPDUs to exhaust switch CPU and keep the topology in constant recalculation, without needing to become root at all.
- All three exploit the same root cause: no authentication on BPDUs. But they achieve different goals: one hijacks the topology, one degrades and exposes traffic, and one just breaks availability outright.

Root Bridge Manipulation (Root Bridge Spoofing)

- Overview: an attacker connects a device to an access port and sends BPDUs advertising an artificially low priority, lower than the legitimate root bridge.
- Vulnerability exploited: BPDUs have no authentication, and the election logic always defers to whoever claims the lowest Bridge ID.
- Attack technique: the network accepts the attacker's device as the new root bridge. Every switch recalculates its topology around this new root, forcing traffic that previously took efficient paths to now flow through the attacker's device.
- Impact: this is a powerful position. The attacker can passively intercept and analyse traffic (man-in-the-middle), or simply keep readvertising to force repeated topology recalculation, causing instability and denial of service across the whole switched network.

Topology Change (TC) Flooding / Spoofing

- Overview: switches normally send a Topology Change Notification (TCN) BPDU whenever a port transitions (e.g. comes up or goes down), so the network can react to a genuine change quickly.
- Vulnerability exploited: like all BPDUs, TCNs carry no authentication. Any device can send one, and every switch that hears it reacts as if a real topology change occurred.
- Attack technique: the attacker floods the network with fabricated TCN BPDUs (or rapidly toggles a port). Each one causes every switch to shorten its MAC address table aging time drastically, flushing entries far faster than normal.
- Impact: constant flushing means switches constantly lose track of where devices are, forcing frequent unknown-unicast flooding out every port, just like Week 3's CAM table attack. This degrades performance network-wide and gives the attacker the same passive eavesdropping opportunity, without ever needing to become the root bridge.

STP Denial of Service (DoS)

- Overview: rather than trying to win the root bridge election, the attacker simply floods the network with a high volume of BPDUs: configuration BPDUs, TCNs, or both.
- Vulnerability exploited: every switch must process every BPDU it receives on every port to keep running the STP algorithm. By default there's no rate limit and no check on how often legitimate-looking BPDUs are allowed to arrive.
- Attack technique: sustained high-rate BPDU flooding consumes switch CPU cycles just processing the flood, and if the BPDUs vary their claimed priority, they can keep forcing the spanning tree to recalculate over and over.
- Impact: switches spend their processing capacity on STP recalculation instead of forwarding traffic, and the network can be kept in a near-permanent reconverging state, a network-wide denial of service that doesn't require ever becoming root bridge.

STP Mitigations

Six features, each protecting a different assumption

The Six Mitigations, at a Glance

Feature	Protects against
Root Guard	A rogue or misconfigured switch claiming the root bridge role (Root Bridge Manipulation)
BPDU Guard	Any BPDU arriving at all on a port that should only ever have an end device attached, including fabricated TCNs (TC Flooding)
BPDU Filter	The same scenario as BPDU Guard, addressed by silently discarding BPDUs instead of shutting the port down
Loop Guard	A loop forming because a blocking port stops receiving BPDUs (e.g. a unidirectional link failure)
BackboneFast	Slow convergence after an indirect link failure elsewhere in the network
UplinkFast	Slow convergence after an access switch's direct uplink to its root port fails

Root Guard

- Deployed on: designated ports of a switch that should never become a root port. Typically distribution/core switch ports facing downstream access switches.
- How it works: if the port receives a BPDU claiming a superior (lower) priority than the current root, the port is moved into a root-inconsistent state and stops forwarding, instead of honouring the claim.
- Effect: an attacker (or misconfigured switch) downstream simply can't become the root bridge from that port, no matter what priority they advertise.
- This directly stops the STP manipulation attack from the previous slide, at the point closest to where an attacker would plug in.

BPDU Guard

- Deployed on: access ports connecting to end-user devices. Typically paired with PortFast, which skips the Listening/Learning states for ports known to have only a single host attached.
- How it works: if any BPDU at all is received on a BPDU Guard-protected port, the port is immediately shut down (err-disabled).
- Effect: legitimate end devices (PCs, printers) never send BPDUs, so this is a safe assumption. Any BPDU appearing here is inherently suspicious and the port is simply taken offline.
- This is usually the very first line of defence against an attacker plugging a rogue switch into a wall port intended for a PC.

BPDU Filter

- Deployed on: the same PC-facing access ports as BPDU Guard, as an alternative approach, or on any port where STP participation should be excluded entirely.
- How it works: the port neither sends nor processes BPDUs at all. Any BPDU that arrives is silently discarded; the port stays up and keeps forwarding normally, with no error-disable action taken.
- Key difference from BPDU Guard: BPDU Guard treats an incoming BPDU as a violation and shuts the port down, a visible, logged response. BPDU Filter simply ignores it and carries on, with no detection and no alert generated.
- Evaluating the trade-off (this is exactly the LO2 skill): BPDU Filter is weaker as a detective control, since nothing is logged and a rogue device could sit there indefinitely unnoticed. But it still stops that device from actually influencing the topology, since its BPDUs are discarded rather than acted on.

Loop Guard

- Deployed on: root ports and alternate/backup (non-designated) ports. Typically on point-to-point links between distribution/core switches.
- How it works: normally, a blocking port only transitions to forwarding after it stops hearing BPDUs. Loop Guard adds a check: if BPDUs stop arriving unexpectedly (e.g. a unidirectional link failure), the port is put into a loop-inconsistent state instead of forwarding.
- Effect: prevents exactly the scenario where a failed link direction tricks STP into thinking a path is safe to unblock, when a loop would actually form.
- This complements Root Guard: Root Guard stops a bad root claim, Loop Guard stops loops from link-failure edge cases.

BackboneFast & UplinkFast

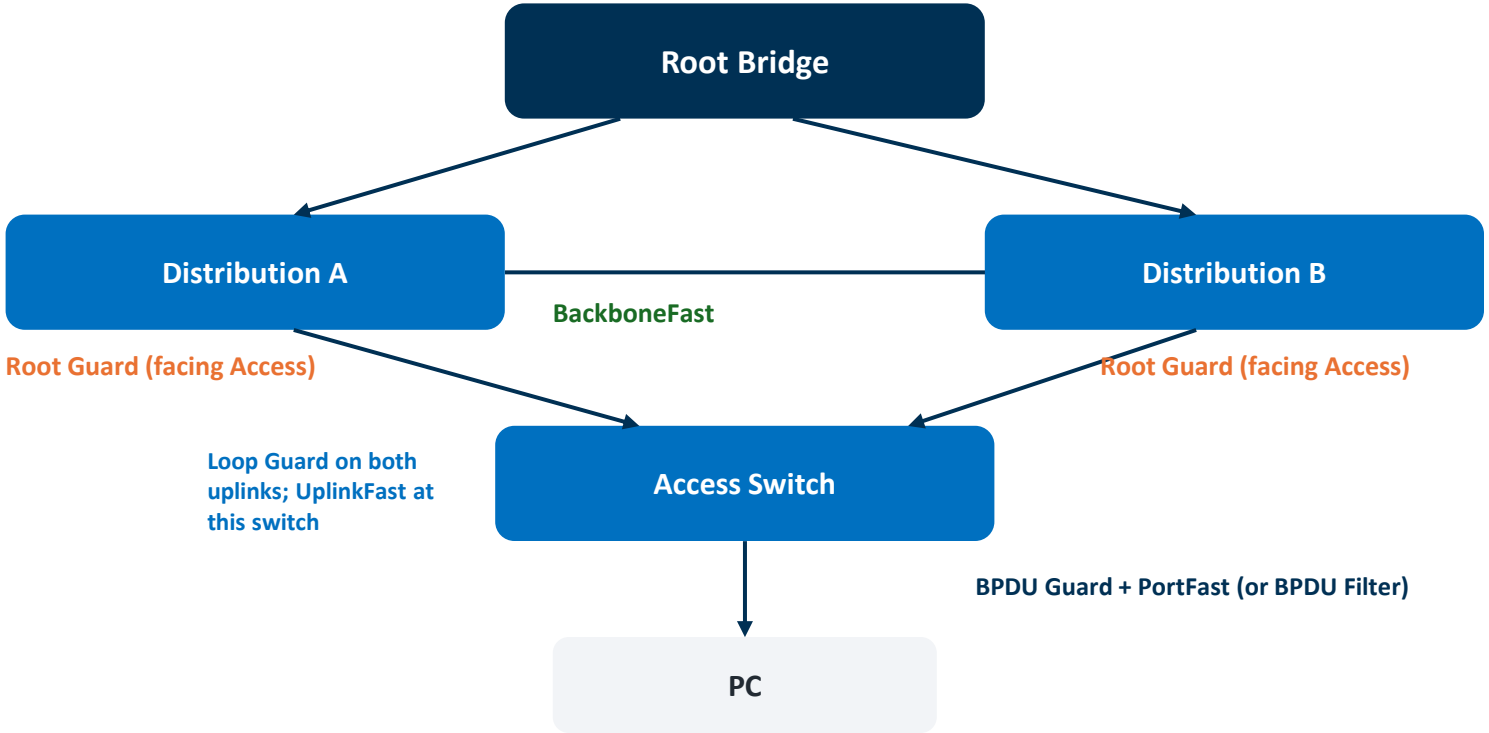
- BackboneFast: deployed on distribution/core switches. Speeds up convergence specifically for indirect link failures (a failure elsewhere in the backbone, not on a switch's own directly connected link), by actively querying for alternate paths instead of just waiting out the Max Age timer.
- UplinkFast: deployed at the switch level on access-layer switches with redundant uplinks. When an access switch's root port (its direct uplink) fails, UplinkFast immediately transitions a previously blocked backup uplink to forwarding, rather than progressing through the normal STP timers.
- Both were Cisco's answer to classic STP's slow convergence, before RSTP built fast convergence into the protocol itself. You'll still find them in older or mixed-version deployments.

Where to Configure Each Mitigation

Mitigation	Where to configure it
Root Guard	Distribution/core switch ports facing downstream access switches: designated ports that must never become root ports
BPDU Guard	Access ports connecting to end-user devices, paired with PortFast
BPDU Filter	Access ports connecting to end-user devices, as an alternative to BPDU Guard, or any port meant to be excluded from STP entirely
Loop Guard	Root ports and alternate/backup ports, typically the point-to-point links between distribution/core switches
BackboneFast	A switch-wide setting on distribution/core switches, not tied to a single port
UplinkFast	A switch-wide setting on access-layer switches that have redundant uplinks, not tied to a single port

Most of these are per-port; BackboneFast and UplinkFast are switch-wide settings instead.

Where Each Mitigation Actually Lives



Root Guard sits closest to where an untrusted claim could enter; BPDU Guard or BPDU Filter sit at the edge, on end-device ports; Loop Guard and UplinkFast protect redundant uplinks; BackboneFast protects the backbone between distribution switches.

Summary

- STP prevents Layer 2 loops by electing a root bridge and blocking all but one path between any two switches. Essential given Ethernet frames have no hop limit.
- Root path cost, accumulated hop by hop, decides both a switch's Root Port and which switch wins the Designated Port role on every shared segment.
- The algorithm trusts every BPDU it receives, with no authentication. This is the root cause of every attack covered today.
- Root Bridge Manipulation hijacks the topology itself; TC Flooding degrades and exposes traffic by forcing constant MAC table flushes. Different mechanisms, same underlying trust gap.
- Root Guard, BPDU Guard/Filter, and Loop Guard each protect a different assumption STP makes. Deployed at different points in the topology, they form a layered defence, not a single fix.

This Week's Lab & Tutorial

- Lab (runs in the Week 5 session, based on this week's lectures): configure STP hardening (Root Guard, BPDU Guard, Loop Guard) alongside Layer 3 mitigations on a Packet Tracer topology.
- Tutorial worksheet: study-notes-style questions covering STP fundamentals, the attack, and each mitigation. Use it to prep for the mid-semester test.

References

- Santos & Stuppi, CCNA Security 210-260 Official Cert Guide, 1st Ed., Cisco Press.
- Kaufman, Perlman, Speciner & Perlner, Network Security: Private Communication in a Public World, 3rd Ed., Prentice Hall.
- Cisco Networking Academy, CCNA Security curriculum (Spanning Tree Protocol security chapters).